

Deep Learning – Homework#13-1

Neural Machine Translation

Welcome to your first programming assignment for this week!

You will build a Neural Machine Translation (NMT) model to translate human readable dates ("25th of June, 2009") into machine readable dates ("2009-06-25"). You will do this using an attention model, one of the most sophisticated sequence to sequence models.

Let's load all the packages you will need for this assignment.

```
from keras.layers import Bidirectional, Concatenate, Permute, Dot, Input, LSTM, Multiply
from keras.layers import RepeatVector, Dense, Activation, Lambda
from keras.optimizers import Adam
from keras.utils import to_categorical
from keras.models import load_model, Model
import keras.backend as K
import numpy as np

from faker import Faker
import random
from tqdm import tqdm
from babel.dates import format_date
from nmt_utils import *
import matplotlib.pyplot as plt
%matplotlib inline
```

1 Translating human readable dates into machine readable dates

The model you will build here could be used to translate from one language to another, such as translating from English to Hindi. However, language translation requires massive datasets and usually takes days of training on GPUs. To give you a place to experiment with these models even without using massive datasets, we will instead use a simpler "date translation" task.

The network will input a date written in a variety of possible formats (e.g. "the 29th of August 1958", "03/30/1968", "24 JUNE 1987") and translate them into standardized, machine readable dates (e.g. "1958-08-29", "1968-03-30", "1987-06-24"). We will have the network learn to output dates in the common machine-readable format YYYY-MM-DD.

1.1 Dataset

We will train the model on a dataset of 10000 human readable dates and their equivalent, standardized, machine readable dates. Let's run the following cells to load the dataset and print some examples.

```
m = 10000
dataset, human_vocab, machine_vocab, inv_machine_vocab = load_dataset(m)
```

```
dataset[:10]
```

You've loaded:

- 'dataset': a list of tuples of (human readable date, machine readable date)
- 'human_vocab': a python dictionary mapping all characters used in the human readable dates to an integer-valued index
- 'machine_vocab': a python dictionary mapping all characters used in machine readable dates to an integer-valued index. These indices are not necessarily consistent with 'human_vocab'.
- 'inv_machine_vocab': the inverse dictionary of 'machine_vocab', mapping from indices back to characters.

Let's preprocess the data and map the raw text data into the index values. We will also use $T_x=30$ (which we assume is the maximum length of the human readable date; if we get a longer input, we would have to truncate it) and $T_y=10$ (since "YYYY-MM-DD" is 10 characters long).

```
Tx = 30
Ty = 10
X, Y, Xoh, Yoh = preprocess_data(dataset, human_vocab, machine_vocab, Tx, Ty)

print("X.shape:", X.shape)
print("Y.shape:", Y.shape)
print("Xoh.shape:", Xoh.shape)
print("Yoh.shape:", Yoh.shape)
```

You now have:

- 'X': a processed version of the human readable dates in the training set, where each character is replaced by an index mapped to the character via 'human_vocab'. Each date is further padded to T_x values with a special character (`␣`). 'X.shape = (m, Tx)'
- 'Y': a processed version of the machine readable dates in the training set, where each character is replaced by the index it is mapped to in 'machine_vocab'. You should have 'Y.shape = (m, Ty)'.
- 'Xoh': one-hot version of 'X', the "1" entry's index is mapped to the character thanks to 'human_vocab'. 'Xoh.shape = (m, Tx, len(human_vocab))'
- 'Yoh': one-hot version of 'Y', the "1" entry's index is mapped to the character thanks to 'machine_vocab'. 'Yoh.shape = (m, Tx, len(machine_vocab))'. Here, 'len(machine_vocab) = 11' since there are 11 characters ('-' as well as 0-9).

Lets also look at some examples of preprocessed training examples. Feel free to play with ‘index’ in the cell below to navigate the dataset and see how source/target dates are preprocessed.

```
index = 0
print("Source date:", dataset[index][0])
print("Target date:", dataset[index][1])
print()
print("Source after preprocessing (indices):", X[index])
print("Target after preprocessing (indices):", Y[index])
print()
print("Source after preprocessing (one-hot):", Xoh[index])
print("Target after preprocessing (one-hot):", Yoh[index])
```

2 Neural machine translation with attention

If you had to translate a book’s paragraph from French to English, you would not read the whole paragraph, then close the book and translate. Even during the translation process, you would read/re-read and focus on the parts of the French paragraph corresponding to the parts of the English you are writing down.

The attention mechanism tells a Neural Machine Translation model where it should pay attention to at any step.

2.1 Attention mechanism

In this part, you will implement the attention mechanism presented in the lecture videos. Here is a figure to remind you how the model works. The diagram on the left shows the attention model. The diagram on the right shows what one ”Attention” step does to calculate the attention variables $\alpha^{(t,t')}$, which are used to compute the context variable $context^{(t)}$ for each timestep in the output ($t = 1, \dots, T_y$).

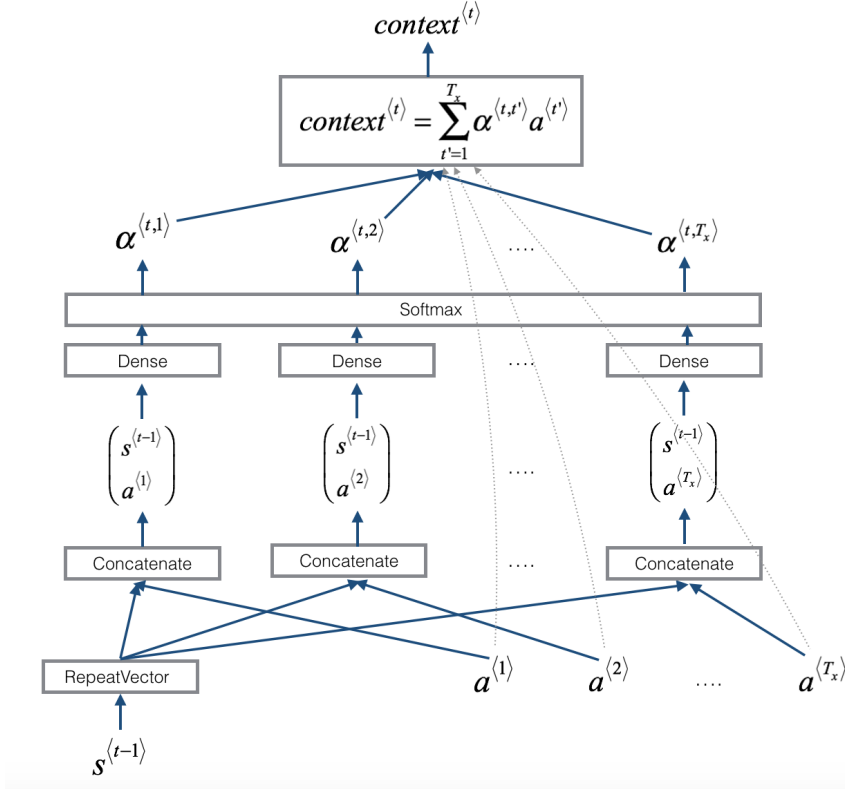
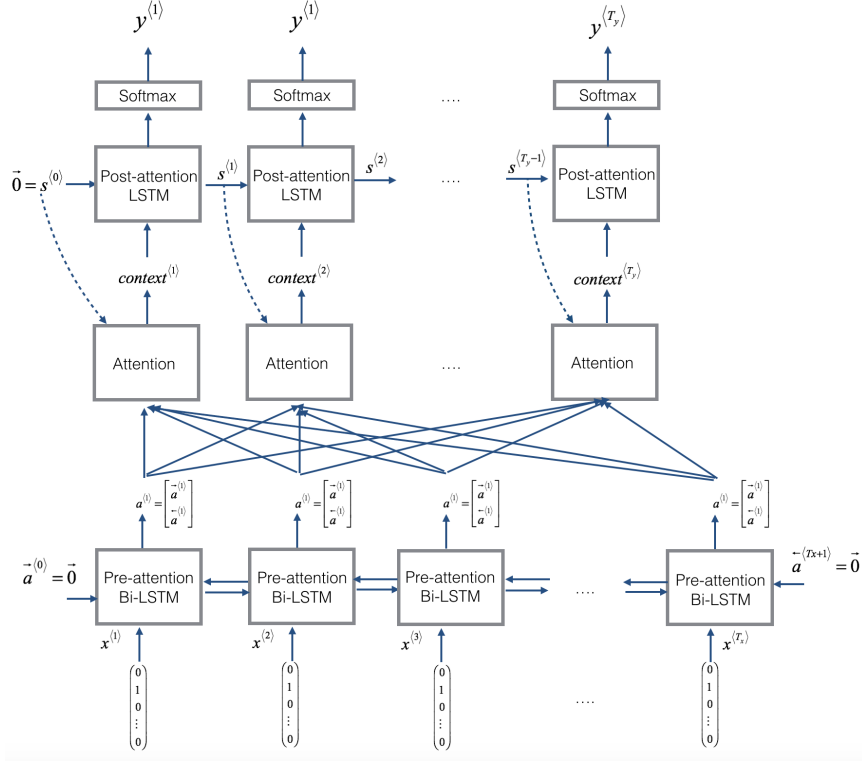


Figure 1: Neural machine translation with attention.

Here are some properties of the model that you may notice:

- There are two separate LSTMs in this model (see diagram on the left). Because the one at the bottom of the picture is a Bi-directional LSTM and comes *before* the attention mechanism, we will call it *pre-attention* Bi-LSTM. The LSTM at the top of the diagram comes *after* the attention mechanism, so we will call it the **post-attention** LSTM. The pre-attention Bi-LSTM goes through T_x time steps; the post-attention LSTM goes through T_y time steps.
- The post-attention LSTM passes $s^{(t)}, c^{(t)}$ from one time step to the next. In the lecture videos, we were using only a basic RNN for the post-activation sequence model, so the state captured by the RNN output activations $s^{(t)}$. But since we are using an LSTM here, the LSTM has both the output activation $s^{(t)}$ and the hidden cell state $c^{(t)}$. However, unlike previous text generation examples (such as Dinosaur in week 1), in this model the post-activation LSTM at time t does will not take the specific generated $y^{(t-1)}$ as input; it only takes $s^{(t)}$ and $c^{(t)}$ as input. We have designed the model this way, because (unlike language generation where adjacent characters are highly correlated) there isn't as strong a dependency between the previous character and the next character in a YYYY-MM-DD date.
- We use $a^{(t)} = [\vec{a}^{(t)}; \overleftarrow{a}^{(t)}]$ to represent the concatenation of the activations of both the forward-direction and backward-directions of the pre-attention Bi-LSTM.
- The diagram on the right uses a 'RepeatVector' node to copy $s^{(t-1)}$'s value T_x times, and then 'Concatenation' to concatenate $s^{(t-1)}$ and $a^{(t)}$ to compute $e^{(t,t')}$, which is then passed through a softmax to compute $\alpha^{(t,t')}$. We'll explain how to use 'RepeatVector' and 'Concatenation' in Keras below.

Lets implement this model. You will start by implementing two functions: 'one_step_attention()' and 'model()'.

1. 'one_step_attention()': At step t , given all the hidden states of the Bi-LSTM ($[a^{<1>}, a^{<2>}, \dots, a^{<T_x>}]$) and the previous hidden state of the second LSTM ($s^{<t-1>}$), 'one_step_attention()' will compute the attention weights ($[\alpha^{<t,1>}, \alpha^{<t,2>}, \dots, \alpha^{<t,T_x>}]$) and output the context vector (see Figure 1 (right) for details):

$$context^{<t>} = \sum_{t'} \alpha^{<t,t'>} a^{<t'>} \quad (1)$$

Note that we are denoting the attention in this homework $context^{(t)}$. In the lecture videos, the context was denoted $c^{(t)}$, but here we are calling it $context^{(t)}$ to avoid confusion with the (post-attention) LSTM's internal memory cell variable, which is sometimes also denoted $c^{(t)}$.

2. 'model()': Implements the entire model. It first runs the input through a Bi-LSTM to get back $[a^{<1>}, a^{<2>}, \dots, a^{<T_x>}]$. Then, it calls 'one_step_attention()' T_y times ('for' loop). At each iteration of this loop, it gives the computed context vector $c^{<t>}$ to the second LSTM, and runs the output of the LSTM through a dense layer with softmax activation to generate a prediction $\hat{y}^{<t>}$.

Exercise: Implement ‘one_step_attention()’. The function ‘model()’ will call the layers in ‘one_step_attention()’ T_y using a for-loop, and it is important that all T_y copies have the same weights. I.e., it should not re-initialize the weights every time. In other words, all T_y steps should have shared weights. Here’s how you can implement layers with shareable weights in Keras:

1. Define the layer objects (as global variables for examples).
2. Call these objects when propagating the input.

We have defined the layers you need as global variables. Please run the following cells to create them. Please check the Keras documentation to make sure you understand what these layers are: [RepeatVector()], [Concatenate()], [Dense()], [Activation()], [Dot()].

```
# Defined shared layers as global variables
repeater = RepeatVector(Tx)
concatenator = Concatenate(axis=-1)
densor1 = Dense(10, activation = "tanh")
densor2 = Dense(1, activation = "relu")
activator = Activation(softmax, name='attention_weights') # We are using a custom softmax(
    axis = 1) loaded in this homework
dotor = Dot(axes = 1)
```

Now you can use these layers to implement ‘one_step_attention()’. In order to propagate a Keras tensor object X through one of these layers, use ‘layer(X)’ (or ‘layer([X,Y])’ if it requires multiple inputs.), e.g. ‘densor(X)’ will propagate X through the ‘Dense(1)’ layer defined above.

```
# GRADED FUNCTION: one_step_attention

def one_step_attention(a, s_prev):
    """
    Performs one step of attention: Outputs a context vector computed as a dot product of
        the attention weights
        "alphas" and the hidden states "a" of the Bi-LSTM.

    Arguments:
    a -- hidden state output of the Bi-LSTM, numpy-array of shape (m, Tx, 2*n_a)
    s_prev -- previous hidden state of the (post-attention) LSTM, numpy-array of shape (m,
        n_s)

    Returns:
    context -- context vector, input of the next (post-attention) LSTM cell
    """

    ### START CODE HERE ###
    # Use repeater to repeat s_prev to be of shape (m, Tx, n_s) so that you can
        concatenate it with all hidden states "a" (~ 1 line)
    s_prev = None
    # Use concatenator to concatenate a and s_prev on the last axis (~ 1 line)
    concat = None
```

```

# Use densor1 to propagate concat through a small fully-connected neural network to
  compute the "intermediate energies" variable e. (~1 lines)
e = None
# Use densor2 to propagate e through a small fully-connected neural network to compute
  the "energies" variable energies. (~1 lines)
energies = None
# Use "activator" on "energies" to compute the attention weights "alphas" (~ 1 line)
alphas = None
# Use dotor together with "alphas" and "a" to compute the context vector to be given
  to the next (post-attention) LSTM-cell (~ 1 line)
context = None
### END CODE HERE ###

return context

```

You will be able to check the expected output of ‘one_step_attention()’ after you’ve coded the ‘model()’ function.

Exercise: Implement ‘model()’ as explained in figure 2 and the text above. Again, we have defined global layers that will share weights to be used in ‘model()’.

```

n_a = 32
n_s = 64
post_activation_LSTM_cell = LSTM(n_s, return_state = True)
output_layer = Dense(len(machine_vocab), activation=softmax)

```

Now you can use these layers T_y times in a ‘for’ loop to generate the outputs, and their parameters will not be reinitialized. You will have to carry out the following steps:

1. Propagate the input into a [Bidirectional] [LSTM]
2. Iterate for $t = 0, \dots, T_y - 1$:
 - (a) Call ‘one_step_attention()’ on $[\alpha^{<t,1>}, \alpha^{<t,2>}, \dots, \alpha^{<t,T_x>}]$ and $s^{<t-1>}$ to get the context vector $context^{<t>}$.
 - (b) Give $context^{<t>}$ to the post-attention LSTM cell. Remember pass in the previous hidden-state $s^{<t-1>}$ and cell-states $c^{<t-1>}$ of this LSTM using ‘initial_state= [previous hidden state, previous cell state]’. Get back the new hidden state $s^{<t>}$ and the new cell state $c^{<t>}$.
 - (c) Apply a softmax layer to $s^{<t>}$, get the output.
 - (d) Save the output by adding it to the list of outputs.
3. Create your Keras model instance, it should have three inputs (“inputs”, $s^{<0>}$ and $c^{<0>}$) and output the list of “outputs”.

```

# GRADED FUNCTION: model

```

```

def model(Tx, Ty, n_a, n_s, human_vocab_size, machine_vocab_size):
    """
    Arguments:
    Tx -- length of the input sequence
    Ty -- length of the output sequence
    n_a -- hidden state size of the Bi-LSTM
    n_s -- hidden state size of the post-attention LSTM
    human_vocab_size -- size of the python dictionary "human_vocab"
    machine_vocab_size -- size of the python dictionary "machine_vocab"

    Returns:
    model -- Keras model instance
    """

    # Define the inputs of your model with a shape (Tx,)
    # Define s0 and c0, initial hidden state for the decoder LSTM of shape (n_s,)
    X = Input(shape=(Tx, human_vocab_size))
    s0 = Input(shape=(n_s,), name='s0')
    c0 = Input(shape=(n_s,), name='c0')
    s = s0
    c = c0

    # Initialize empty list of outputs
    outputs = []

    ### START CODE HERE ###

    # Step 1: Define your pre-attention Bi-LSTM. Remember to use return_sequences=True. (~
    1 line)
    a = None

    # Step 2: Iterate for Ty steps
    for t in range(Ty):

        # Step 2.A: Perform one step of the attention mechanism to get back the context
        vector at step t (~ 1 line)
        context = None

        # Step 2.B: Apply the post-attention LSTM cell to the "context" vector.
        # Don't forget to pass: initial_state = [hidden state, cell state] (~ 1 line)
        s, _, c = None

        # Step 2.C: Apply Dense layer to the hidden state output of the post-attention
        LSTM (~ 1 line)
        out = None

        # Step 2.D: Append "out" to the "outputs" list (~ 1 line)
        None

    # Step 3: Create model instance taking three inputs and returning the list of outputs.
    (~ 1 line)
    model = None

    ### END CODE HERE ###

```



```
return model
```

Run the following cell to create your model.

```
model = model(Tx, Ty, n_a, n_s, len(human_vocab), len(machine_vocab))
```

Let's get a summary of the model to check if it matches the expected output.

```
model.summary()
```

Expected Output:

Here is the summary you should see

```
Total params:    185,484
Trainable params: 185,484
Non-trainable params: 0
bidirectional_1's output shape (None, 30, 128)
repeat_vector_1's output shape (None, 30, 128)
concatenate_1's output shape (None, 30, 256)
attention_weights's output shape (None, 30, 1)
dot_1's output shape (None, 1, 128)
dense_2's output shape (None, 11)
```

As usual, after creating your model in Keras, you need to compile it and define what loss, optimizer and metrics you want to use. Compile your model using 'categorical_crossentropy' loss, a custom [Adam] [optimizer] ('learning rate = 0.005', $\beta_1 = 0.9$, $\beta_2 = 0.999$, 'decay = 0.01') and '['accuracy']' metrics:

```
### START CODE HERE ### (~2 lines)
opt = None
None
### END CODE HERE ###
```

The last step is to define all your inputs and outputs to fit the model:

- You already have X of shape ($m = 10000$, $T_x = 30$) containing the training examples.
- You need to create 's0' and 'c0' to initialize your 'post_activation_LSTM_cell' with 0s.
- Given the 'model()' you coded, you need the "outputs" to be a list of 11 elements of shape (m, T_y). So that: 'outputs[i][0], ..., outputs[i][Ty]' represent the true labels (characters) corresponding to the i^{th} training example ('X[i]'). More generally, 'outputs[i][j]' is the true label of the j^{th} character in the i^{th} training example.

```
s0 = np.zeros((m, n_s))
c0 = np.zeros((m, n_s))
outputs = list(Yoh.swapaxes(0,1))
```

Let's now fit the model and run it for one epoch.

```
model.fit([Xoh, s0, c0], outputs, epochs=1, batch_size=100)
```

While training you can see the loss as well as the accuracy on each of the 10 positions of the output. The table below gives you an example of what the accuracies could be if the batch had 2 examples:

True labels 1	1	9	9	5	-	1	2	-	0	4
Predictions 1	1	9	9	5	-	1	0	-	0	5
True labels 2	1	9	6	8	-	0	1	-	0	4
Predictions 2	1	9	7	8	-	0	3	-	0	4
Index	1	2	3	4	5	6	7	8	9	10
Accuracy	1.0	1.0	0.5	1.0	1.0	1.0	0.0	1.0	1.0	0.5

Thus, `dense_2_acc_8: 0.89` means that you are predicting the 7th character of the output correctly 89% of the time in the current batch of data.

We have run this model for longer, and saved the weights. Run the next cell to load our weights. (By training a model for several minutes, you should be able to obtain a model of similar accuracy, but loading our model will save you time.)

```
model.load_weights('models/model.h5')
```

You can now see the results on new examples.

```
EXAMPLES = ['3 May 1979', '5 April 09', '21th of August 2016', 'Tue 10 Jul 2007', ' '
            'Saturday May 9 2018', 'March 3 2001', 'March 3rd 2001', '1 March 2001']
for example in EXAMPLES:

    source = string_to_int(example, Tx, human_vocab)
    source = np.array(list(map(lambda x: to_categorical(x, num_classes=len(human_vocab)),
                             source))).swapaxes(0,1)
    prediction = model.predict([source, s0, c0])
    prediction = np.argmax(prediction, axis = -1)
    output = [inv_machine_vocab[int(i)] for i in prediction]

    print("source:", example)
    print("output:", ''.join(output))
```

You can also change these examples to test with your own examples. The next part will give you a better sense on what the attention mechanism is doing—i.e., what part of the input the network is paying attention to when generating a particular output character.

3 Visualizing Attention

Since the problem has a fixed output length of 10, it is also possible to carry out this task using 10 different softmax units to generate the 10 characters of the output. But one advantage of the attention model is that each part of the output (say the month) knows it needs to depend only on a small part of the input (the characters in the input giving the month). We can visualize what part of the output is looking at what part of the input.

Consider the task of translating "Saturday 9 May 2018" to "2018-05-09". If we visualize the computed $\alpha^{(t,t')}$ we get this:

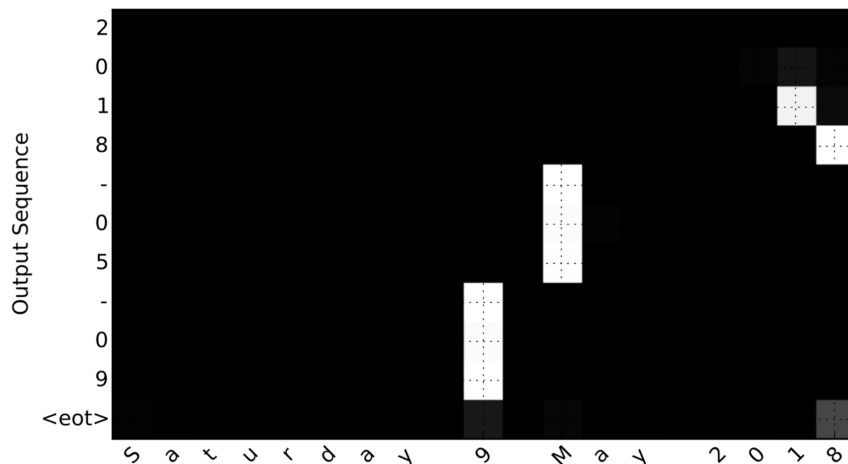


Figure 2: Full Attention Map.

Notice how the output ignores the "Saturday" portion of the input. None of the output timesteps are paying much attention to that portion of the input. We see also that 9 has been translated as 09 and May has been correctly translated into 05, with the output paying attention to the parts of the input it needs to to make the translation. The year mostly requires it to pay attention to the input's "18" in order to generate "2018."

3.1 Getting the activations from the network

Lets now visualize the attention values in your network. We'll propagate an example through the network, then visualize the values of $\alpha^{(t,t')}$.

To figure out where the attention values are located, let's start by printing a summary of the model.

```
model.summary()
```

Navigate through the output of ‘model.summary()’ above. You can see that the layer named ‘attention_weights’ outputs the ‘alphas’ of shape (m, 30, 1) before ‘dot_2’ computes the context vector for every time step $t = 0, \dots, T_y - 1$. Lets get the activations from this layer.

The function ‘attention_map()’ pulls out the attention values from your model and plots them.

```
attention_map = plot_attention_map(model, human_vocab, inv_machine_vocab, "Tuesday 09 Oct  
1993", num = 7, n_s = 64)
```

In the generated plot you can observe the values of the attention weights for each character of the predicted output. Examine this plot and check that where the network is paying attention makes sense to you.

In the date translation application, you will observe that most of the time attention helps predict the year, and hasn’t much impact on predicting the day/month.

Congratulations!

You have come to the end of this assignment

Here’s what you should remember from this homework:

- Machine translation models can be used to map from one sequence to another. They are useful not just for translating human languages (like **French**->**English**) but also for tasks like date format translation.
- An attention mechanism allows a network to focus on the most relevant parts of the input when producing a specific part of the output.
- A network using an attention mechanism can translate from inputs of length T_x to outputs of length T_y , where T_x and T_y can be different.
- You can visualize attention weights $\alpha^{(t,t')}$ to see what the network is paying attention to while generating each output.

Congratulations on finishing this assignment! You are now able to implement an attention model and use it to learn complex mappings from one sequence to another.